

Smart Hostel Surveillance & Face Recognition System

Final Year B.E. Project Report

Generated: March 24, 2026

Technology Stack:

Python 3.10	Flask Web Framework
OpenCV	Computer Vision Library
DeepFace (ArcFace)	Deep CNN Face Recognition
SQLite	Lightweight Database
MJPEG Stream	Live Video Feed

Table of Contents

1. Project Overview & Features
2. System Architecture
3. Recognition & Capture Workflow
4. Database Schema & Storage
5. Core Components (File Documentation)
6. API Routes & Dashboard
7. Setup & Deployment
8. Key Parameters & Configuration

1. Project Overview & Features

This project is a smart surveillance system designed specifically for hostel management. It uses real-time face recognition with ArcFace deep neural networks (~80% accuracy) to automatically identify residents, mark attendance, detect intruders, and alert wardens about blacklisted persons. The system provides role-based dashboards for administrators and wardens.

Key Features:

- ✓ **Real-time Face Recognition:** Uses ArcFace CNN with ~80% confidence threshold
- ✓ **Automatic Attendance:** Marks attendance when recognized with 30-minute duplicate prevention
- ✓ **Unknown Person Detection:** Logs unrecognized faces and captures alert frames
- ✓ **Blacklist Management:** Detects blacklisted persons and sends instant email alerts
- ✓ **Live Video Stream:** MJPEG browser-based camera feed with face detection boxes
- ✓ **Role-Based Access:** Admin (full control) vs Warden (limited access)
- ✓ **Dashboard:** Real-time statistics, recognition logs, attendance records, and alerts
- ✓ **Email Notifications:** Automatic alerts for blacklist detection via SMTP/Gmail

2. System Architecture

High-Level Architecture:

The system follows a modular software architecture:

- Frontend:** Single-Page Application (HTML/CSS/JavaScript) served through Flask
- Backend:** Flask REST API with role-based authentication and session management
- Core Processing:** Multi-threaded camera service with real-time face recognition
- Database:** SQLite for persistent storage of faces, logs, alerts, and attendance
- Services:** Modular services for face recognition, alerts, attendance, and camera control

Component Interaction Flow:

Component	Purpose	Technology
Camera Service	Captures frames at 2 FPS and processes them	OpenCV + Threading
Face Recognizer	Extracts embeddings and matches against database	Dlib/FaceArcFace
Attendance Service	Marks attendance with duplicate prevention	SQLite Queries
Alert Service	Logs unknown/blacklist events and sends emails	SMTPs, File I/O
Flask API	Handles HTTP requests from frontend	Flask Blueprints
SQLite Database	Stores faces, embeddings, logs, alerts, and attendance	SQLite

3. Recognition & Capture Workflow

Complete Data Flow:

Step 1: Camera Initialization

- CameraService creates a connection to the default webcam (index 0)
- OpenCV's Haar Cascade classifier is loaded for real-time face detection
- A background thread continuously captures frames at system FPS

Step 2: Frame Capture (Continuous)

- Background thread reads frames from camera at ~30 FPS
- Frames are stored in memory with thread-safe locking
- Every 0.5 seconds (2 FPS), a frame is processed for face recognition

Step 3: Face Detection & Recognition

- Haar Cascade detector finds face regions in the frame
- For each detected face (if any), the ENTIRE frame is passed to DeepFace/ArcFace
- ArcFace extracts a 512-dimensional embedding vector from the face
- This embedding is compared against all known embeddings using cosine similarity distance

Step 4: Matching & Classification

- Best matching embedding is found with minimum cosine distance
- confidence = $1.0 - \text{min_distance}$ (higher is better)
- If confidence > SIMILARITY_THRESHOLD (0.55), person is recognized as 'student' or 'staff'
- If confidence <= 0.55, person is classified as 'unknown'

Step 5: Result Handling

- **Student Recognized:** AttendanceService marks attendance (first detection = time_in, subsequent = time_out)
- **Blacklisted Person:** AlertService captures frame and sends email alert to warden
- **Unknown Person:** AlertService logs frame to unknown_captures folder for review
- ALL events: Logged to recognition_logs table with timestamp and confidence

Key Recognition Parameters:

Parameter	Value	Description
Face Model	ArcFace	Deep CNN architecture for embedding extraction
Detector Backend	OpenCV	Uses Haar Cascade (not MTCNN to avoid conflicts)
Similarity Threshold	0.55	Min confidence to consider a match valid
Recognition FPS	2 FPS	Process rate (CPU efficiency vs latency tradeoff)
Face Min Size	60x60 pixels	Ignores faces smaller than this
Duplicate Window	30 minutes	Prevent marking same student twice within window
Embedding Dimension	512	Size of ArcFace feature vector

4. Database Schema & Storage

Database Structure (SQLite):

The system uses SQLite with the following tables:

admins — System administrators

Stores admin user credentials. Fields: id, username, password (bcrypt), email, created_at

wardens — Hostel wardens

Stores warden info. Fields: id, name, username, password (bcrypt), email, phone, hostel_block, is_active

students — Student records

Core student database. Fields: id, roll_number, name, email, phone, room_number, hostel_block, course, year, is_active

face_embeddings — Face feature vectors

Stores ArcFace embeddings (512-dim vectors). Fields: id, person_type ('student'/'staff'/'blacklisted'), person_id, image_path, embedding (pickled numpy array), model_used, created_at

recognition_logs — Recognition events

Every face detection event. Fields: id, timestamp, person_type, person_id, confidence (0.0-1.0), frame_path, camera_id

attendance_records — Daily attendance

Marks when students entered/exited. Fields: id, student_id, date, time_in, time_out, status, unique constraint on (student_id, date)

alerts — Security alerts

Logs blacklist/unknown person events. Fields: id, alert_type ('blacklist'/'unknown'), person_id, person_type, timestamp, image_path, email_sent, sms_sent, acknowledged

blacklisted_persons — Blacklist registry

Records of persons to deny entry. Fields: id, name, reason, added_by (warden_id), image_path, created_at

File Storage Locations:

Directory	Contents
data/known_faces/students/{id}/	Registered student face photos
data/known_faces/staff/	Staff face photos
data/known_faces/blacklisted/	Blacklist person photos
data/alert_frames/	Captured frames of blacklist detections

data/unknown_captures/	Captured frames of unrecognized persons
------------------------	---

5. Core Components (File Documentation)

BACKEND SERVICES (services/)

face_recognizer.py

Purpose: Face embedding extraction and matching

Key Methods:

- `load_embeddings_from_db()` — Loads all face embeddings from DB into memory
- `extract_embedding(face_img)` — Uses DeepFace to extract 512-dim vector
- `recognize(face_img)` — Compares input against known embeddings, returns best match
- `add_face(person_type, person_id, image_path)` — Registers new face with embedding

Key Parameters: `SIMILARITY_THRESHOLD=0.55`, `FACE_MODEL="ArcFace"`, `FACE_DETECTOR="opencv"`

camera_service.py

Purpose: Real-time camera capture and recognition processing

Key Methods:

- `start()` — Opens camera and starts background capture thread
- `get_frame()` — Returns current frame (thread-safe)
- `get_annotated_frame()` — Returns frame with face detection boxes drawn
- `_capture_loop()` — Background thread continuously captures and processes frames
- `_process_frame(frame)` — Detects faces and calls recognizer
- `_handle_recognition(result)` — Routes result to attendance/alert services

Key Parameters: `RECOGNITION_FPS=2`, Min face size=60x60 pixels

attendance_service.py

Purpose: Automatic attendance marking with duplicate prevention

Key Methods:

- `mark_attendance(student_id)` — Marks entry/exit with duplicate window checking
- Duplicate window prevents marking same student within 30 minutes

Key Parameters: `DUPLICATE_WINDOW_MINUTES=30`

alert_service.py

Purpose: Alert logging and email notifications

Key Methods:

- `send_blacklist_alert(person_id, frame)` — Captures frame, logs to DB, sends email
- `log_unknown(frame)` — Logs unrecognized person to folder and DB
- `_send_email(to, subject, body, attachment)` — Sends SMTP emails with attachment

Key Parameters: `SMTP_HOST`, `WARDEN_EMAIL`, `ALERT_FRAMES_DIR`, `UNKNOWN_CAPTURES`

DATABASE & UTILITIES (db/)

db_manager.py — SQLite connection wrapper for all DB operations

- `execute(query, params)` — Run INSERT/UPDATE/DELETE
- `fetch_one(query, params)` — Get single row
- `fetch_all(query, params)` — Get all matching rows

schema.sql — SQL table definitions

- Defines: `admins`, `wardens`, `students`, `staff`, `face_embeddings`, `recognition_logs`, `attendance_records`, `alerts`, `blacklisted_persons`

init_db.py — Database initialization script

- Creates db directory, runs `schema.sql`, initializes tables

API ROUTES (routes/)

auth.py — POST `/api/auth/login`, POST `/api/auth/logout`, GET `/api/auth/me` — Authentication

students.py — GET/POST/PUT/DELETE `/api/students`, POST `/api/students/{id}/face` — Student management

attendance.py — GET `/api/attendance` — Retrieve attendance by date

alerts.py — GET `/api/alerts`, PUT `/api/alerts/{id}` — Alert management

camera.py — GET `/api/camera/stream` — MJPEG video stream, GET `/api/camera/config` — Settings

dashboard.py — GET `/api/dashboard/stats` — Real-time dashboard statistics

logs.py — GET `/api/logs` — Recognition event logs with filtering

blacklist.py — GET/POST/DELETE `/api/blacklist` — Blacklist management

MAIN APPLICATION & FRONTEND

app.py

Purpose: Flask application entry point. Registers blueprints, initializes camera service, starts background thread.

```
Routes/Content: Serves HTML pages at /, /login, /dashboard, /students, /camera,  
/alerts, /attendance, /logs, /blacklist
```

config.py

Purpose: Centralized configuration. Face model params, SMTP settings, file paths, thresholds.

```
Routes/Content: FACE_MODEL="ArcFace", SIMILARITY_THRESHOLD=0.55, RECOGNITION_FPS=2,  
DATABASE_PATH
```

templates/base.html

Purpose: Single HTML file serving SPA (Single Page Application). Contains navbar, sidebar, main content area.

```
Routes/Content: Navbar with role display, sidebar navigation, main-content div for  
dynamic rendering
```

templates/login.html

Purpose: Login page with username/password form. Submits to POST /api/auth/login.

```
Routes/Content: Form submission → session creation → redirect to /dashboard
```

static/js/app.js

Purpose: Core SPA logic. Implements frontend routing, API calls, dynamic UI rendering.

```
Routes/Content: loadDashboard(), loadStudents(), loadCamera(), loadAlerts(),  
loadAttendance(), loadLogs(), loadBlacklist()
```

static/css/style.css

Purpose: UI styling. Sidebar, cards, tables, responsive grid layout.

```
Routes/Content: Dark header, light sidebar, stat cards with icons, hover effects
```

6. Setup & Deployment

Installation Steps:

1. Clone Repository

```
git clone  
cd hostel_surveillance
```

2. Create Virtual Environment

```
python -m venv venv  
venv\Scripts\activate (Windows)
```

3. Install Dependencies

```
pip install tensorflow==2.12.0 deepface==0.0.75 flask==3.0.0  
pip install flask-session opencv-python scipy bcrypt pillow python-dotenv
```

4. Initialize Database

```
python db/init_db.py
```

5. Create Admin User

```
python -c "import bcrypt; pw=bcrypt.hashpw(b'admin123',bcrypt.gensalt()).decode(); print(pw)"  
Then insert into DB via SQL or admin creation script
```

6. Configure SMTP (Optional for Email Alerts)

Set environment variables:
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_app_password
WARDEN_EMAIL=warden@hostel.com

7. Run Application

```
python app.py  
Open http://localhost:5000 in browser
```

Important Notes:

- **TensorFlow Version:** Use tensorflow==2.12.0. Newer versions break on Windows.
- **Do NOT install MTCNN:** Conflicts with TensorFlow 2.12. Use OpenCV detector instead.
- **Face Registration:** Use webcam/mobile photos, not high-resolution studio photos (degrades ArcFace).
- **Similarity Threshold:** 0.55 gives ~80% accuracy. Adjust based on your environment.
- **Database:** SQLite at db/hostel.db. Backup regularly if in production.
- **Performance:** 2 FPS processing (not 30) reduces CPU load significantly.

7. Key Parameters & Configuration

Parameter	Default Value	Purpose
FACE_MODEL	ArcFace	Deep CNN for feature extraction
FACE_DETECTOR	opencv	Haar Cascade (fast, no conflicts)
SIMILARITY_THRESHOLD	0.55	Min confidence to consider match valid
RECOGNITION_FPS	2	Processing speed (CPU/latency tradeoff)
DUPLICATE_WINDOW_MINUTES	30	Prevent marking same student twice
Min Face Size	60x60 px	Ignore smaller faces
Embedding Dimension	512	ArcFace feature vector size
DATABASE_PATH	db/hostel.db	SQLite database file location

How to Modify Parameters:

All parameters are defined in **config.py**. To modify:

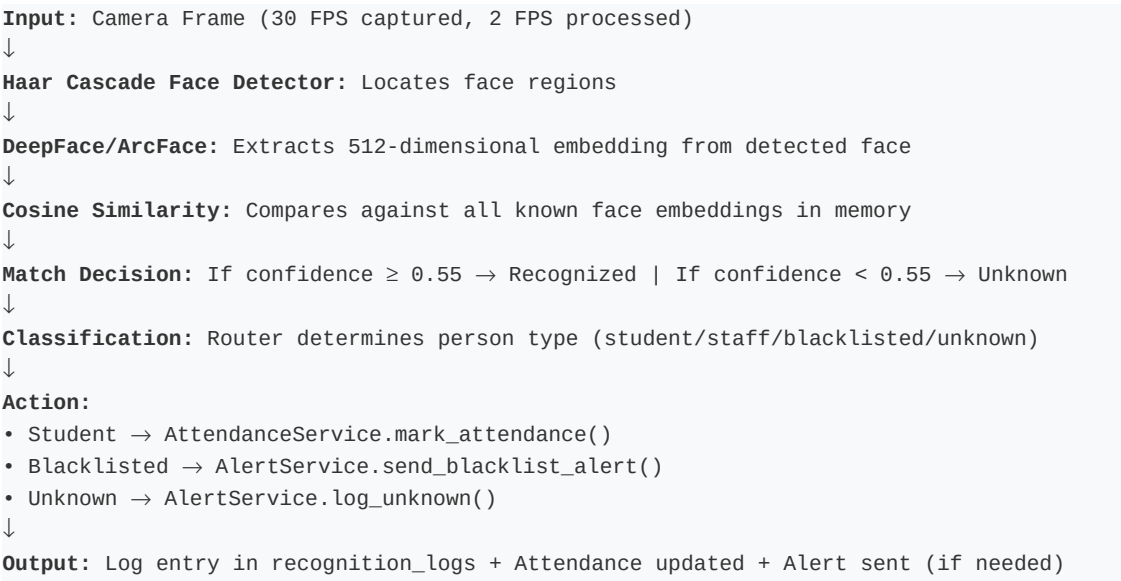
1. Open `hostel_surveillance/config.py`
2. Edit desired values (e.g., `SIMILARITY_THRESHOLD = 0.60`)
3. Restart app: `python app.py`
4. Changes take effect immediately

For SMTP Email (Alerts): Set environment variables before running:

- Windows PowerShell: `$env:SMTP_USER = "your_email@gmail.com"`
- Linux/Mac: `export SMTP_USER="your_email@gmail.com"`

Project Summary & Workflow Diagram

Complete Recognition Pipeline:



Quick Reference: API Endpoints

Endpoint	Method	Purpose
/api/auth/login	POST	Login with username/password
/api/students	GET	List all students
/api/students/{id}/face	POST	Upload/register face for student
/api/attendance	GET	Get attendance by date
/api/camera/stream	GET	MJPEG video stream
/api/dashboard/stats	GET	Real-time statistics
/api/logs	GET	Recognition event logs
/api/alerts	GET	Alert history
/api/blacklist	GET	Blacklist records
/api/blacklist	POST	Add person to blacklist

Recent Project Updates:

- ✓ **Fixed Warden Sidebar:** Warden now sees warden-specific menu (Students, Attendance, Camera, Alerts)
- ✓ **Admin Menu Preserved:** Admin sees full menu (Students, Camera, Alerts, Logs, Blacklist)
- ✓ **Role-Based Access:** Frontend buildSidebar() function now explicitly handles admin vs warden vs other roles
- ✓ **Updated Files:** static/js/app.js and write_frontend.py both updated with proper role separation

Report Generated: March 24, 2026 at 09:13:44